

实验一：线性规划算法

姓名：郝思粤 学号：PB23151779

2026 年 1 月 24 日

1 实验目的与要求

1. 实现线性规划的标准型转换及预处理。
2. 利用 QR 分解检测并处理约束矩阵的秩，确保满秩。
3. 实现大 M 法以解决初始可行基问题。
4. 实现单纯形法迭代过程，并应用 Bland 规则解决退化与循环问题。
5. 统计并分析算法在不同规模数据下的运行效率。

2 算法设计与模块实现

2.1 模块 0：预处理与标准型转换

在线性规划求解前,我首先对输入数据进行前置处理,对应代码中的 `_preprocess()`。该模块的核心目标是：在进入大 M 法与单纯形迭代之前，尽量消除明显矛盾/冗余，并把 RHS 规范到适合初始化的形式（尤其是保证 $b \geq 0$ ）。

核心代码

```
1 # 1) 检查全为0的行
2 zero_rows = np.all(self.A == 0, axis=1)
3 if np.any(zero_rows):
4     if np.any(self.b[zero_rows] != 0):
5         self.status = "LP has no feasible point"
6         return False
7     else:
8         self.A = self.A[~zero_rows]
9         self.b = self.b[~zero_rows]
10        self.m = self.A.shape[0]
```

```

11
12 # 2) 修复：确保 RHS 非负（等式行整体乘 -1）
13 neg_rhs = self.b < 0
14 if np.any(neg_rhs):
15     self.A[neg_rhs, :] *= -1.0
16     self.b[neg_rhs] *= -1.0

```

代码实现原理与功能解释 第一段是零行检测：如果某个约束行 $a_i = 0$ ，但 $b_i \neq 0$ ，那就是 $0 = b_i$ 直接矛盾，提前判无可行解；如果 $b_i = 0$ ，该行是恒等式 $0 = 0$ ，属于冗余约束，删掉即可。第二段是 RHS 负号修正：当存在 $b_i < 0$ 时，把对应等式整行乘 -1 ，可行域不变，但能把 RHS 调整为非负。这一步对大 M 法很关键，因为人工变量初始化通常取 $a = b$ ，若 b 为负会导致“初始基解不可行”，迭代会非常不稳定。

2.2 模块 1：矩阵秩检查 (QR 分解)

为了保证后续基矩阵可逆，约束矩阵 A 需要行满秩。本实验在 `_check_full_rank_qr()` 中，先判矛盾再删冗余：如果 $\text{rank}(A) < \text{rank}([A|b])$ 则无解；若不矛盾但秩亏，则通过带列主元 QR 找出可保留的独立约束行。

核心代码

```

1 rank_A = np.linalg.matrix_rank(self.A)
2 Augmented = np.hstack([self.A, self.b.reshape(-1, 1)])
3 rank_Augmented = np.linalg.matrix_rank(Augmented)
4
5 if rank_A < rank_Augmented:
6     self.status = "LP_has_no_feasible_point"
7     return False
8
9 B = self.A.T # n x m
10 tol = 1e-10
11 from scipy.linalg import qr as scipy_qr
12 Q, R, piv = scipy_qr(B, pivoting=True, mode="economic")
13 diag_R = np.abs(np.diag(R)) if R.size else np.array([])
14 r = int(np.sum(diag_R > tol))
15
16 if r < self.m:
17     keep_rows = np.sort(piv[:r])
18     self.A = self.A[keep_rows, :]
19     self.b = self.b[keep_rows]
20     self.m = self.A.shape[0]

```

代码实现原理与功能解释 前两行计算 $\text{rank}(A)$ 与 $\text{rank}([A|b])$ ，若出现前者小于后者，就意味着等式系统本身矛盾（典型例子：同一左端但右端不同），此时不必进入后续流程，直接返回无可行解。若不矛盾但秩亏，则按实验要求构造 $B = A^T$ 并做带主元 QR：主元序列 `piv` 会把“独立程度更高”的列（对应原 A 的行）排在前面。通过判断 $|R_{ii}|$ 是否大于阈值 `tol` 得到数值秩 r ，然后只保留 `piv[:r]` 对应的约束行，从而删除冗余行并把系统变成满秩，避免基矩阵奇异导致的迭代异常。

2.3 模块 2：初始可行基解 (大 M 法)

如果无法直接构造初始可行基，则引入人工变量使用大 M 法。该部分在 `solve()` 中完成：把人工变量拼到决策变量后面，用 $[A \ I]$ 构造扩展约束，并设置目标系数为 M 以惩罚人工变量。初始化时把人工变量作为基变量，得到显式可行基解。

核心代码

```
1 M = 100000.0
2
3 # 扩展 c: [c_1, ..., c_n, M, ..., M]
4 c_extended = np.concatenate([self.c, np.full(self.m, M)])
5
6 # 扩展 A: [A | I]
7 A_extended = np.hstack([self.A, np.eye(self.m)])
8
9 # 初始基变量索引 (人工变量)
10 basis_indices = list(range(self.n, self.n + self.m))
```

代码实现原理与功能解释 大 M 法的核心是“用人工变量保证一开始就有基可行解”。扩展后约束为 $[A \ I][x; a] = b$ ，把人工变量 a 作为初始基变量时，基矩阵就是 I ，因此初始基解直接是 $a = b, x = 0$ 。这也解释了为什么模块 0 要把 b 修成非负：否则人工变量初值会为负，从而破坏基可行性。目标函数中给人工变量乘上一个很大的 M ，迭代会倾向于把人工变量压到 0；如果最后仍有人工变量为正，说明原问题本身无可行解。

2.4 模块 3：单纯形法迭代与 Bland 规则

在单纯形迭代部分，本实现采用单纯形表 (tableau) 形式做 pivot 更新，并用 Bland 规则解决退化情况下的循环风险。每轮迭代：先由检验数选择入基变量，再用最小比率检验选择出基行，最后做主元归一化与消元更新单纯形表。

核心代码

```
1 # 1) 入基：找 reduced cost < 0 的列（最小化）
2 reduced_costs = tableau[0, 1:]
3 candidates = np.where(reduced_costs < -1e-9)[0]
4 if len(candidates) == 0:
5     break
6 entering_idx = candidates[0] # Bland
7
8 # 2) 出基：最小比率检验
9 pivot_col = tableau[1:, entering_idx + 1]
10 rhs = tableau[1:, 0]
11 ratios, row_indices = [], []
12 for i in range(self.m):
13     if pivot_col[i] > 1e-9:
14         ratios.append(rhs[i] / pivot_col[i])
15         row_indices.append(i)
16 if not ratios:
17     self.status = "unbounded_LP"
18     return self.status
19
20 min_ratio = min(ratios)
21 min_ratio_indices = [row_indices[i] for i, r in enumerate(ratios)
22                      if abs(r - min_ratio) < 1e-9]
23
24 # 3) Pivot：主元归一化 + 消元
25 pivot_row_idx_in_tableau = leaving_row_local + 1
26 pivot_element = tableau[pivot_row_idx_in_tableau, entering_idx + 1]
27 tableau[pivot_row_idx_in_tableau, :] /= pivot_element
28 for i in range(self.m + 1):
29     if i != pivot_row_idx_in_tableau:
30         factor = tableau[i, entering_idx + 1]
31         tableau[i, :] -= factor * tableau[pivot_row_idx_in_tableau,
32                                           :]
```

代码实现原理与功能解释 第一步通过检验数 (reduced cost) 判断是否还能改进：对最小化问题，只要存在 $r_j < 0$ 就说明把对应变量增加一点可以降低目标值，于是它成为入基候选。这里用 Bland 规则直接取“下标最小的候选”作为入基变量，避免在退化时出现循环。第二步做最小比率检验：只在入基列系数为正的行里计算 $\theta = b_i/a_{ij}$ ，取最小者保持可行；若所有系数都 ≤ 0 ，说明可以无限增大入基变量而不破坏可行性，同时还能

继续改进目标，因此判为无界。第三步用 pivot 更新单纯形表：把主元行归一化，再对其他行消元，相当于在表上完成一次换基更新，不需要显式求 B^{-1} 。这一套实现能直接对应讲义里的单纯形法流程，也方便调试与观察每轮迭代的变化。

3 实验结果验证

3.1 基础测试案例

以下展示代码在不同情形下的输出结果（均来自程序运行输出）：

1. 有唯一最优解：

```
>>> 测试 1: 标准有解 (Min -3x1 - 2x2)
[原始问题]
  A: [[2, 1, 1, 0, 0], [1, 1, 0, 1, 0], [1, 0, 0, 0, 1]]
  b: [100, 80, 40]
  c: [-3, -2, 0, 0, 0]
[结果] 状态: Optimal
[目标值] Optimal Z: -180.0
[解向量] x: [20. 60.  0.  0. 20.]
```

图 1: 有唯一解

测试 1 中，程序返回 Optimal，最优目标值为 $Z^* = -180.0$ ，对应解为

$$x = [20, 60, 0, 0, 20].$$

2. 有冗余约束 (秩亏)：

```
>>> 测试 2: 冗余约束测试 (QR分解应移除多余行)
[原始问题]
  A: [[1, 1, 1], [2, 2, 2]]
  b: [10, 20]
  c: [1, 1, 0]
[原始维度] m=2
[处理后维度] m=1 (若减少说明成功移除冗余)
[结果] 状态: Optimal
[目标值] Optimal Z: -0.0
[解向量] x: [ 0.  0. 10.]
```

图 2: 有冗余约束

测试 2 中原始约束行数 $m = 2$ ，处理后变为 $m = 1$ ，QR 秩检测删掉了冗余行。程序最终仍返回 Optimal，并给出解

$$x = [0, 0, 10],$$

目标值为 -0.0 (数值上等价于 0)。

3. 无可行解 (Infeasible):

```
>>> 测试 3: 无可行解 (应输出: LP has no feasible point)
[原始问题]
  A: [[1, 1], [1, 1]]
  b: [10, 20]
  c: [2, 1]
[结果] 状态: LP has no feasible point
```

图 3: 无可行解

测试 3 中秩检查阶段发现 $\text{rank}(A) < \text{rank}([A|b])$, 程序返回 LP has no feasible point。

4. 无界解 (Unbounded):

```
>>> 测试 4: 无界解 (应输出: unbounded LP)
[原始问题]
  A: [[1, -1, 1]]
  b: [10]
  c: [-1, 0, 0]
[结果] 状态: unbounded LP
```

图 4: 无界解

测试 4 中, 程序在比值检验阶段发现入基列无法找到正系数行进行约束, 判定无界并返回 unbounded LP。

3.2 Bland 规则验证 (循环案例)

本实验额外加入书上的“循环基解”例子:

$$\min -4x_1 - x_2, \text{ s.t. } \begin{cases} -x_1 + 2x_2 \leq 4 \\ 2x_1 + 3x_2 \leq 12 \\ x_1 - x_2 \leq 3 \\ x_1, x_2 \geq 0 \end{cases}$$

```

=== 书上循环基解例子 ===
状态: Optimal
用时: 0.000132s
目标值: -18.0
解向量: [4.2 1.2 5.8 0. 0. ]

```

图 5: 课本上循环例子验证

程序输出为 `Optimal`，说明在该退化/循环风险较高的例子上也能正常收敛。结合实现，关键就在于 Bland 规则对入基/出基的并列处理，避免了在有限多个基之间来回兜圈。

4 算法效率统计分析

本次实验对规模 $(m, n) = (10, 20), (20, 40), \dots$ 的问题进行了测试。每个规模随机生成 20 组数据，**仅统计返回最优解 (Optimal) 的样本**，无可行/无界等情况直接跳过不计。实验输出的统计结果如下表所示（方差为程序打印的“方差”）：

表 1: 不同规模下的求解时间统计（20 次随机测试，跳过非最优样本）

规模 ($m \times n$)	成功样本数	平均时间 (s)	时间方差
10 × 20	10/20	0.0006	0.000000
20 × 40	8/20	0.0028	0.000000
30 × 60	14/20	0.0087	0.000001
40 × 80	11/20	0.0216	0.000008
50 × 100	9/20	0.0654	0.001071
60 × 120	13/20	0.0850	0.000078
70 × 140	8/20	0.1341	0.000926
80 × 160	8/20	0.2108	0.000326
90 × 180	9/20	0.3071	0.002198
100 × 200	10/20	0.4478	0.003311
110 × 220	10/20	0.6600	0.006082
120 × 240	15/20	0.8437	0.003210
130 × 260	13/20	1.1519	0.007757
140 × 280	9/20	1.5668	0.021416
150 × 300	7/20	1.9629	0.009962
160 × 320	2/20	2.1299	0.008875

可以看出，随着规模增大，平均运行时间整体呈上升趋势，并且在较大规模时方差也明显变大。一方面是单纯形法 pivot 次数在不同随机实例之间差异较大；另一方面大 M 法引入的巨大惩罚系数在大规模下更容易带来数值不稳定，从而导致部分样本无法顺利得到 Optimal（成功率在 $m \geq 150$ 时明显下降）。

同时，由于单纯形法在规模较大时的时间复杂度明显上升，导致规模超过 160×320 之后，算法求解的迭代次数超过了该算法在设计时的最高迭代次数，导致无法得出解的情况，这也是为什么平均时间会在超过 160×320 之后陡然下降的原因，并不是算法求解的时间降低了，反而是因为时间太长了。

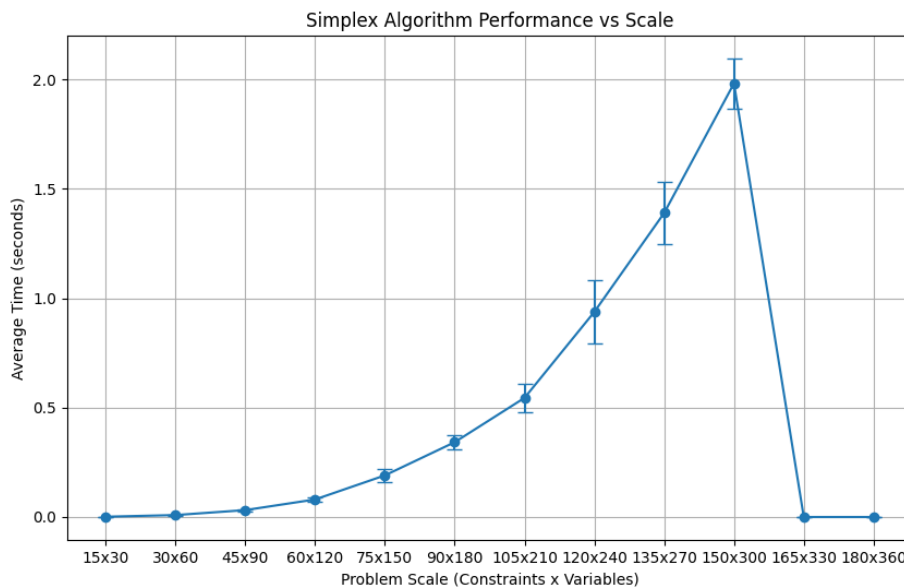


图 6: 求解时间随问题规模的变化趋势（均值 $\pm$ 标准差）

5 总结与心得

这次实验相当于把讲义里单纯形法的流程完整实现了一遍。比较关键的几点体会：

- 预处理很重要。最开始性能测试规模一大就会“卡很久”，后来发现很多随机样本的 b 会出现负数，导致大 M 法的初始基解其实不满足非负性，后面 pivot 会变得非常慢。把 $b_i < 0$ 的等式行统一乘 -1 之后，运行稳定很多。
- QR 的秩处理能明显减少问题规模。冗余约束那一组测试里， $m = 2$ 能直接删到 $m = 1$ ，否则后面基矩阵不可逆会直接影响迭代。
- Bland 规则在退化/循环例子里很有用。虽然这次没有专门实现“关闭 Bland 的版本”做对照，但从书上循环例子的结果来看，至少开启 Bland 后能正常收敛，不会出现一直换基停不下来的情况。

- 大 M 法实现简单，但数值稳定性确实一般。规模变大时成功样本数会下降，说明大 M 常数的选择、以及浮点误差都会影响最终能否稳定得到最优。后续如果要改进，我觉得两阶段法会更稳一些；同时也可以考虑用修正单纯形 (revised simplex) 或 LU 更新来减少每次 pivot 的开销。